

EFRM Rule Optimization Agent

Complete project context, architecture decisions, current status, first-PR MVP and continuation plan

Category	Details
Purpose	Enable another team member to understand the project and continue implementation without additional explanation.
Current implementation priority	First-PR MVP: API trigger, optional LLM Planner, fixed-plan fallback, one background worker and LangGraph Plan-and-Execute flow.
Long-term product scope	Enterprise Rule Optimization Agent for evidence-based recommendations on transaction-monitoring rules.
Authority	Recommendation only. No production rule write, retirement or deletion.
Technology baseline	Python, FastAPI, LangGraph and PostgreSQL. No Temporal, Java or Spring Boot for Phase 1.
Status date	31 August 2026

One-sentence summary The project will build a separate Agentic Platform service that eventually analyzes rules linked to finalized false-positive cases, but the immediate first PR only proves the reusable API, LLM planning, background execution, LangGraph state flow and minimum persistence.

Internal working document

This document records agreed decisions and explicitly separates completed design work from unimplemented functionality.

Contents

1. Executive handover summary
2. Product problem and intended outcome
3. Final product scope and authority
4. Enterprise target architecture
5. Complete future Rule Optimizer workflow
6. Data, snapshot and analytics design
7. LLM, RAG and deterministic processing boundaries
8. Shared Agentic Platform and database design
9. Current project status and decisions already completed
10. Immediate first-PR MVP scope
11. First-PR execution story
12. LangGraph Plan-and-Execute behavior
13. Minimum first-PR database model
14. First-PR APIs, worker and configuration
15. First-PR acceptance criteria and test plan
16. Implementation sequence and repository structure
17. Deferred work and subsequent PR roadmap
18. Open inputs and risks
19. Decision log
20. Handover checklist and glossary

1. Executive handover summary

The Rule Optimization Agent belongs to a separate shared Agentic Platform, not to the existing EFRM Rule Engine. The platform will host multiple agents, beginning with the Rule Optimizer and Executive Copilot. The Rule Optimizer is a background analytical agent; it does not create transaction alerts and it does not directly change production rules.

The long-term Rule Optimizer starts from finalized false-positive cases. It traces each case to its alerts, match records and exact historical rule versions, builds a bounded historical snapshot, runs deterministic analytics, optionally uses an LLM to explain validated findings, and produces a recommendation for human review.

The immediate implementation priority is narrower. The first PR is an MVP of the generic Plan-and-Execute runtime. It must receive an API request, create or load an execution plan, invoke LangGraph through one background worker, execute demonstration steps, persist status and checkpoint state, and return a final result. It must not yet access EFRM data or implement Rule Optimization tools.

Critical separation Enterprise architecture describes where the product is going. First-PR MVP scope describes what must be coded now. Team members must not implement future analytics or EFRM integration inside the first PR.

1.1 Current status at handover

Item	Status	Notes
Product documents	Reviewed	PRD, database flow reference, platform diagram, colleague review and updated Agentic DB design were analyzed.
Enterprise architecture	Designed	Components, data boundaries, workflow, tool families, LLM boundary and database flow are documented.
Phase 1 authority	Finalized	Recommendation only; humans and existing governance own production changes.
First-PR MVP	Finalized for implementation	API, optional LLM Planner, fixed plan, one worker, LangGraph and five application tables.
Production code	Not confirmed as implemented	This handover does not claim that the runtime or Rule Optimizer has been coded.
EFRM integration	Deferred	No Case/Alert/Rule service contract or production read connection is confirmed.

2. Product problem and intended outcome

The existing EFRM platform already contains transaction-monitoring rules, processes transaction or device facts, generates alerts and supports cases and investigations. Rules behave as binary match/no-match controls. A rule can be technically correct but still contribute to a case that is finally closed as a false positive.

The business requirement changed from continuously analyzing every rule to prioritizing only the rules connected to finalized false-positive cases. This creates a focused and explainable starting signal.

2.1 Business question

- Which exact rules and historical rule versions contributed to a finalized false-positive case?
- Is the rule generating useful alerts or excessive unnecessary alerts?
- Has the rule's effectiveness or stability changed over time?
- Could its threshold or scenario logic be reviewed to reduce unnecessary alerts without unsupported risk claims?
- What evidence should a human reviewer see before deciding whether to change the rule?

2.2 Intended long-term output

The primary output is a human-review recommendation, not an automatically applied rule change.

Output text	Meaning
Rule reference	Exact logical rule and analyzed historical version.

Output field	Meaning
Issue	Observed problem such as false-positive concentration, decay, unstable behavior or overlap.
Recommended action	Review threshold, review condition, investigate overlap, collect evidence, no change or consider retirement.
Reason	Clear explanation of why the action is suggested.
Evidence	Metrics, periods, denominators, lineage references, hashes and limitations.
Expected impact	Directional expectation such as lower alert volume; not an unsupported production guarantee.
Authority	HUMAN_REVIEW_REQUIRED.

3. Final product scope and authority

3.1 Long-term capabilities

- Rule health analysis: volume, outcome quality, contribution, stability, capacity and evidence strength.
- Rule decay detection: configurable comparison of baseline and recent periods with minimum sample and persistence gates.
- Threshold analysis: sensitivity around existing thresholds and alert-volume constraints.
- Scenario recommendation: evidence-bounded suggestions to review conditions, grouping or overlap.
- Historical backtesting: future Candidate + Test mode, subject to rule compiler and simulator integration.
- Risk leakage reporting: only evidence-supported internal gaps; do not claim true fraud recall without an independent truth source.
- Human-review recommendation with evidence, limitations and immutable history.

3.2 Phase 1 - Recommendation Only

Phase 1 analyzes existing rule evidence and produces a recommendation. It does not create an executable candidate in the Rule Engine, compile DRL, write rule configuration or deploy a rule.

Production boundary The agent cannot create, modify, retire or delete production rules. Any actual change must be performed by authorized humans through existing EFRM governance.

3.3 Future configurable mode

Mode	Behavior	Phase
Recommendation Only	Analyze evidence and produce human-readable advice. No candidate execution is required.	Phase 1
Candidate + Test	Create a candidate representation, verify the simulator with the current rule, backtest the candidate and compare results.	Deferred

3.4 Institution and channel neutrality

The agent is not designed for one institution, transaction type, channel or device type. Institution-specific outcome mappings, time windows, thresholds, metric policies, constraints and permissions must eventually be configuration, not hard-coded logic.

The supplied data is test/sample data. Its row counts and distributions must not be used for production sizing or performance assumptions.

4. Enterprise target architecture

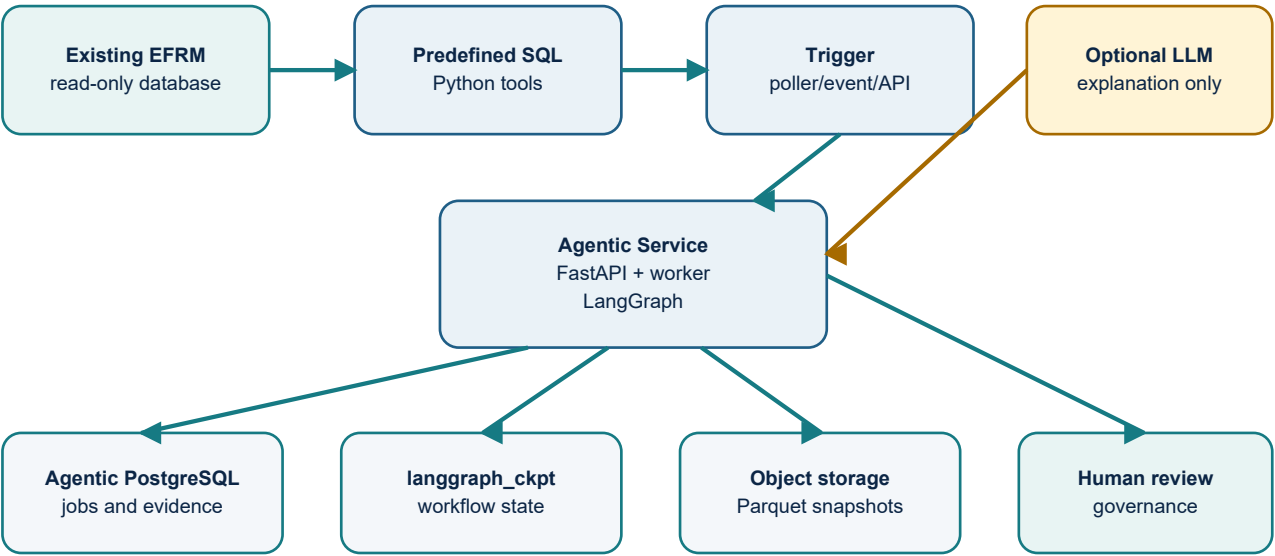


Figure 1. Long-term architecture and ownership boundaries

4.1 Component responsibilities

Component	Responsibility
Existing EFRM	Authoritative source for cases, alerts, decisions, transactions, devices, matches, rules and permissions.
Read-only data tools	Predefined parameterized SQL handlers for current Phase 1 data access. No model-generated SQL.
FastAPI	Trigger, status, recommendation and future review endpoints.
Background worker	Claims queued work, prepares the plan, creates the graph run, calls LangGraph and saves final status.
LangGraph	Controls state, node order, conditional routing, checkpoints and recovery.
Python analytics	Produces official deterministic metrics and findings.
LLM gateway	Planner in generic platform flows and explanation in the future optimizer; no unrestricted database access.
Agentic PostgreSQL	Application configuration, executions, artifacts, findings, recommendations and audit.
Object storage	Large partitioned Parquet snapshots and large artifacts.
Human review	Reviews recommendations and uses existing governance for any production change.

4.2 Technology decisions

Decision	Technology Decision	Rationale
Language	Python	LangGraph, analytics and agent tooling are implemented in one stack.
API	FastAPI	Async internal API and clear schema contracts.
Workflow	LangGraph	Shared state, conditional routing and PostgreSQL checkpoints.
Application database	PostgreSQL	Relational execution, evidence, recommendation and audit records.
Vector retrieval	pgvector later; Qdrant/Pinecone possible later	RAG is supportive, not required for core Rule Optimizer truth.
Files	Approved object/analytical store; SFTP only for transport	Large snapshots should not be stored in checkpoints or operational tables.
Excluded	No Temporal, Java or Spring Boot for Phase 1	The selected shared platform is Python and LangGraph.

5. Complete future Rule Optimizer workflow

1. Trigger: Detect a newly finalized false-positive case through polling, a future event or an authorized API.
2. Verify case: Re-read the authoritative final case decision. The final case decision takes precedence over alert-level decisions.
3. Get alerts: Resolve all source-aware alerts connected to the case.
4. Get rule matches: Follow alert to result/fact/match records and capture exact rule and version identity.
5. Select rules: Deduplicate matches while preserving multiple historical versions of the same logical rule.
6. Resolve configuration: Retrieve historical and current rule configuration, groups, thresholds and Metric Definition Custom SQL references.
7. Plan snapshot: Select bounded institution/rule/fact scope, cutoff, windows, cohorts, fields and partitions.
8. Build snapshot: Execute approved SQL, aggregate or page large data, and write large results to partitioned Parquet.
9. Validate data: Check lineage, version resolution, completeness, sample sufficiency, freshness and reproducibility.
10. Run analytics: Calculate health, decay, threshold behavior, overlap and evidence-supported gaps using Python and SQL.
11. Build strategy: Apply deterministic institution policy to choose allowed recommendation directions.
12. Explain: Optionally ask an LLM to explain only validated findings and limitations.
13. Compose and validate: Create canonical recommendation JSON and reject unsupported claims or stale evidence.
14. Save and review: Persist recommendation, evidence and status for human review. Governance owns production changes.

5.1 Trigger decision

Decision	Rationale
Scheduled SQL poller	Current practical Phase 1 option because service/event contracts are not confirmed.
Event	Preferred future integration if EFRM publishes a durable CASE_FINALIZED event.
API	Authorized manual reanalysis or normalized internal trigger.

5.2 Data access decision

For current Phase 1 design, the optimizer does not assume direct connectivity to Case Service, Alert Service or Rule Service. It uses predefined, parameterized SQL tools against an approved EFRM read-only boundary. Later APIs may implement the same tool contracts without changing the LangGraph workflow.

LLM restriction The LLM does not generate SQL, execute SQL, calculate official metrics or receive unrestricted EFRM database access.

6. Data, snapshot and analytics design

6.1 Meaning of eligible transactions or facts

The snapshot builder does not copy every EFRM transaction for every rule run. It creates a bounded population defined by institution, selected rule/version, fact type, source-aware lineage, configured windows and analysis policy.

Population	Purpose	Context
Rule-evaluated or matched population	Understand volume and outcome behavior for the selected rule.	Point-in-time rule/version context.
False-positive-linked population	Measure the triggering outcome signal.	Final outcome mapping and maturity policy.
Comparable non-match population	Support comparisons only where technically available.	Explicit cohort definition; never invent negatives.
Near-threshold band	Study threshold sensitivity.	Configured band and minimum sample.
Overlap population	Find rules that fired together.	Source-aware match lineage.

6.2 Storage rule

- Small metadata, counts, hashes, findings and recommendations belong in Agentic PostgreSQL.
- Large snapshot rows belong in partitioned Parquet in approved object or analytical storage.
- LangGraph checkpoints store workflow state and references, never the complete analytical snapshot.
- The snapshot manifest stores cutoff, window, schema, query hash, row counts, parts, watermarks and reconstruction information.

6.3 Metric families

Family	Examples	Important limitation
Volume	Match count, alert count, case count and alert rate.	Always include denominator and period.
Outcome quality	False-positive rate, yield and conversion after finalized outcomes.	Do not treat unresolved cases as negatives.
Contribution	Share of alerts/cases where the selected rule contributed.	Deduplicate with match-level lineage.
Stability	Variance, confidence intervals and segment drift.	Require minimum sample and persistence.
Decay	Baseline versus recent deltas and sustained deterioration.	One unusual day is not decay.
Threshold	Outcome and volume sensitivity around current values.	Alert-volume limit is the current confirmed capacity constraint.
Overlap/leakage	Duplicate signals and evidence-supported internal gaps.	Do not claim true recall without independent labels.

7. LLM, RAG and deterministic processing boundaries

7.1 Two different LLM roles

Location	When used	What it may do	What it may not do
Generic Planner	Optional at workflow level; always LLM-based if invoked.	Create a structured plan using allowed stage names.	Invent code, permissions or unauthorized tools.
Optimizer explanation	Future full workflow after validated analytics.	Explain findings, evidence and limitations in readable language.	Access EFRM directly, calculate metrics or approve strategy.

7.2 Planner decision

The Planner component is LLM-only. Invoking it is optional at the workflow level. The current Rule Optimizer may bypass it and use an approved fixed plan. If the Planner is invoked and fails, the execution continues with the approved fixed fallback plan.

```
Workflow configuration
  planner enabled  -> LLM Planner -> validate -> LangGraph
  planner bypassed -> approved fixed plan -> LangGraph
  planner failed   -> record failure -> fixed fallback -> LangGraph
```

7.3 RAG decision

RAG may later provide policy documents, rule-engine documentation, operating procedures and explanations of configuration fields. It is not the source of truth for cases, transactions, matches, outcomes or rule versions. Those remain database/service records.

8. Shared Agentic Platform and database design

8.1 Physical design

Component	Description
Database	One PostgreSQL database named efrm_agentic.
Application schema	agentic - application-owned configuration, execution, evidence and optimizer records.
Checkpoint schema	langgraph_ckpt - runtime-owned checkpoint tables.
EFRM truth	External existing EFRM database/read boundary; read-only to the agent.
Large data	Object/analytical store for Parquet snapshots; SFTP only for approved transport.

8.2 Updated database design status

The reviewed Agentic DB v1.1 contains 91 application tables and four LangGraph checkpoint tables. It is a shared enterprise target schema, not the minimum schema required for the first PR.

8.3 Important database corrections already identified

- Make case-rule lineage source-aware and generic enough for transaction, device and future fact types.
- Allow the same logical rule to appear with multiple historical versions in one case.
- Allow a missing current-rule hash when a current rule was deleted or cannot be resolved.
- Represent Phase 1 strategy as deterministic; keep LLM metadata for explanation, not strategy ownership.
- Add STRATEGY_ITEM to controlled evidence subject types.
- Seed all required tools, stages, policies and bindings before a full workflow can run.

8.4 Important source-system facts

- Decision codes vary by institution and may change; outcome meaning must be mapped by configuration.
- The final case decision takes precedence when alert and case decisions conflict.
- Aggregated_metric is not currently used. Derived metrics are represented through Metric Definition Custom SQL.
- Metric Definition Custom SQL does not yet have confirmed governance controls.
- The existing formal rule representation and JSON-to-DRL compiler exist, but their integration contract is unconfirmed.
- Historical match rows may reference deleted rules; the agent must record missing history and never guess a version.

9. Current project status and decisions already completed

Activity	Details
Product and database review	Existing product flows, schema groups, Rule Engine concepts and shared Agentic DB were reviewed from available documents and sample schema.
Architecture refinement	Separate repository/service, Python/FastAPI/LangGraph stack, read-only EFRM boundary and human-governed authority were finalized.
Workflow design	Trigger-to-recommendation nodes, tool responsibilities, input/output boundaries and LLM location were defined.
Snapshot guidance	Bounded extraction, Parquet storage, manifests and data-quality controls were defined.
Database alignment	Agentic DB v1.1 table flow and required corrections were documented.
First-PR decision	The manager requested Plan-and-Execute architecture before actual optimizer tools; MVP scope was reduced accordingly.

9.1 What has not been completed

- No production Rule Optimizer code implementation is confirmed by this handover.
- No EFRM read-only connection, safe views or production service contracts are confirmed.
- No production-scale historical dataset or sizing evidence is available.
- No final metric thresholds, decay windows, sample-size policy or outcome mapping set is approved.
- No LLM deployment/provider and no production data-egress decision is confirmed.
- No candidate compilation or simulator integration is confirmed.

10. Immediate first-PR MVP scope

First PR objective Prove that the platform can receive an invocation, create or load a plan, execute the plan through LangGraph using one background worker, persist status and recover graph state. No Rule Optimization business step is included.

10.1 Build in this PR

- FastAPI trigger endpoint.
- FastAPI status endpoint.
- One separately running Python background worker.
- Optional LLM Planner component; it always uses an LLM when invoked.
- Approved fixed plan and fixed-plan fallback.
- Small plan validator: valid structure, non-empty steps and allowed stage names.
- Generic LangGraph Plan-and-Execute graph.
- Demonstration handlers only; no EFRM access.
- Five application tables and LangGraph PostgreSQL checkpoints.
- Basic success, bypass, fallback, failure and restart tests.

10.2 Explicitly exclude

- Agent version and workflow version resolution.
- Institution-specific configuration registry beyond basic request ownership.
- Case, alert, transaction, device, match or rule data access.
- Rule health, decay, threshold, overlap or leakage analytics.
- Recommendations, review UI, RAG, candidate rules and simulation.
- Temporal, Kafka, Celery, Java and Spring Boot.

- Complex policy engine, outbox, dead-letter queue and production audit framework.

10.3 MVP configuration

To keep the first PR small, core settings may be stored in environment variables or a simple application configuration file. Normal API callers do not manually select Planner mode.

Setting	Meaning
planner_enabled	If true, call the LLM Planner. If false, load the fixed plan.
planner_model	Configured model deployment used only by the Planner.
fixed_plan_path	Approved JSON plan used for bypass or fallback.
allowed_stage_codes	Small allow-list of demonstration stages.
checkpoint_database_url	Dedicated connection/search path for langgraph_ckpt.

11. First-PR execution story

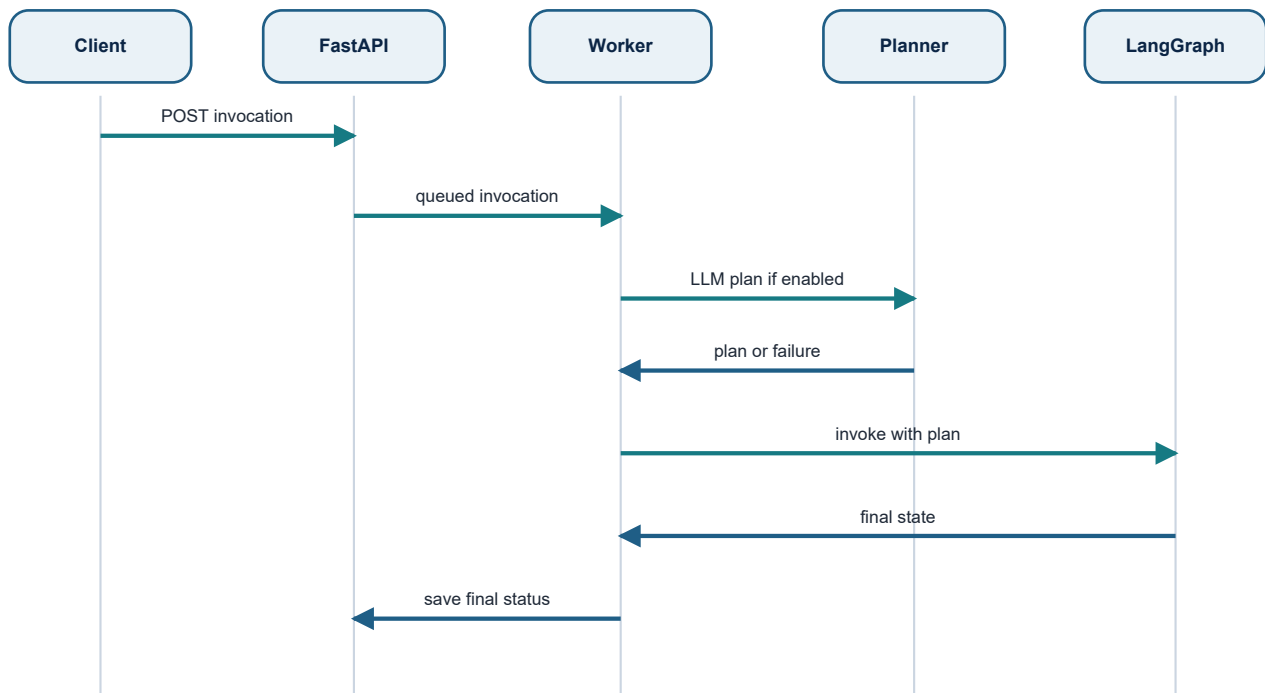


Figure 2. First-PR request, planning and execution sequence

1. Client triggers the API: POST /v1/agent-invocations receives request_id, institution context and a small input object.
2. API creates the invocation: Store agent_invocation with status QUEUED and return HTTP 202 immediately.
3. Worker finds queued work: The one MVP background worker polls for QUEUED records.
4. Worker claims the invocation: Atomically change QUEUED to RUNNING so the same invocation cannot be started twice.
5. Worker reads simple runtime settings: Read planner_enabled, model setting, fixed plan location, graph key and allowed stages from configuration.
6. Plan is prepared: If Planner is enabled, call the LLM. If bypassed, load the fixed plan. If LLM fails, load fixed fallback.
7. Plan is saved: Persist execution_plan and, when the LLM was called, model_execution.
8. Graph run is created: Create graph_run with its LangGraph thread_id and status RUNNING.
9. Initial graph state is built: Put invocation IDs, plan, current index, small context, results and status into state.
10. Worker calls LangGraph: The worker invokes the compiled graph with the initial state and thread_id.
11. LangGraph executes the plan: Load plan, run one handler at a time, update state and route until complete or failed.
12. Checkpoints are written: LangGraph saves recoverable state after graph steps.
13. Final result returns to worker: The graph returns COMPLETED or FAILED state.
14. Worker saves final status: Update graph_run and agent_invocation. The status API can now return the result.

11.1 One worker clarification

The MVP deploys one background worker. 'Worker' and 'background worker' are the same component. The claim operation is still atomic so a restart or future second replica cannot start the same queued invocation twice.

12. LangGraph Plan-and-Execute behavior

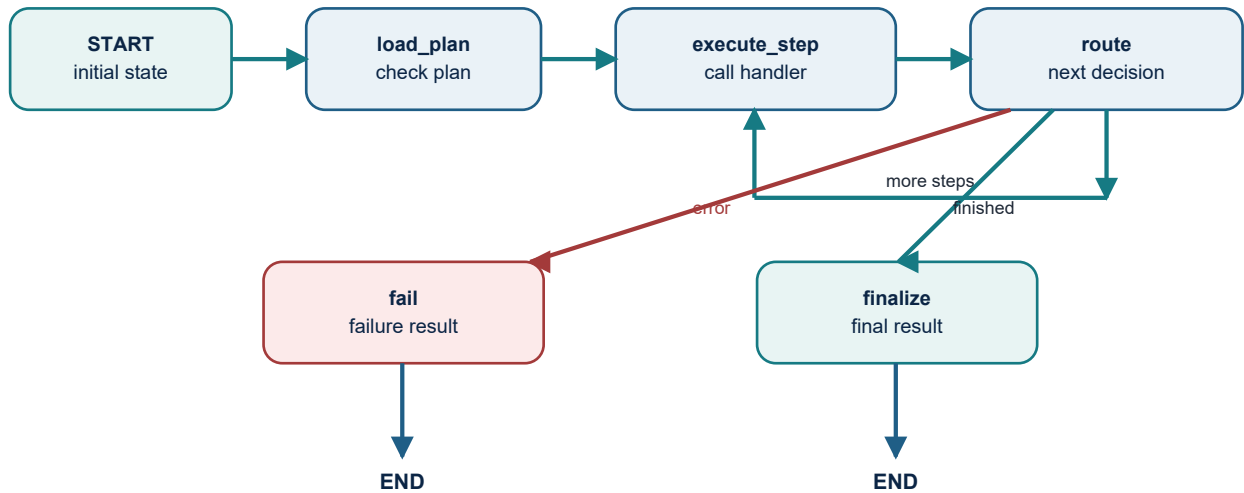


Figure 3. Generic state loop inside LangGraph

12.1 Graph state

State item	Purpose
invocation_id and graph_run_id	Connect state to application records.
plan_id and plan	Identify the saved plan and its ordered steps.
current_step_index	Tell the executor which plan step is next.
current_step_id and stage_code	Identify the step currently running.
context	Small values shared between demonstration handlers.
step_results	Append results from completed steps.
status and error	Represent RUNNING, COMPLETED or FAILED state.
final_result	Return the final graph output.

12.2 Nodes

Node	What happens
load_plan	Confirm the plan exists, has at least one step and uses allowed stage codes.
execute_step	Read current_step_index, select one plan step, find its approved Python handler and execute it.
route_after_step	If failed, go to fail. If more steps exist, loop. If finished, go to finalize.
finalize	Build final result, set COMPLETED and finish the graph.
fail	Build failure result, set FAILED and finish the graph.

12.3 How state moves

1. Worker creates state with current_step_index = 0 and an empty result list.
2. load_plan checks the plan and passes state to execute_step.
3. execute_step reads step 0 and calls the registered demonstration handler.
4. The handler returns a small result and optional context updates.
5. The node appends the result and changes current_step_index to 1.

6. The router sees another step and sends the updated state back to `execute_step`.
7. The loop continues until the index equals the number of plan steps.
8. `finalize` creates the final result and LangGraph returns it to the worker.

Handler safety The LLM chooses from allowed stage codes. It does not supply Python code or arbitrary function names. A registry maps approved stage codes to approved Python handlers.

13. Minimum first-PR database model

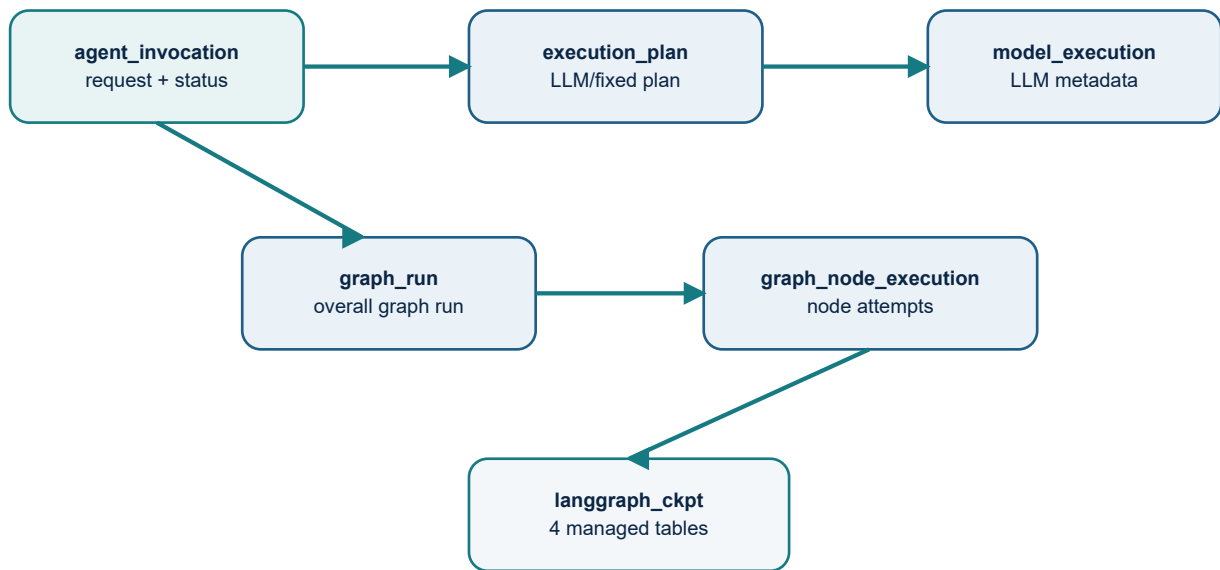


Figure 4. Minimum application records and LangGraph checkpoint relationship

MVP database decision Not all 95 tables from the enterprise Agentic DB are required. First PR uses five application tables plus four LangGraph-managed checkpoint tables.

13.1 Application-managed tables

Table	Columns	Why it is needed
agent_invocation	request_id, institution_id, input, status, correlation ID, result, error and timestamps.	Main record for the API-triggered execution. UNIQUE(request_id) provides basic duplicate protection.
execution_plan	invocation ID, plan source, plan JSON, hash, status and creation time.	Provides the exact LLM, fixed or fallback plan passed to LangGraph.
model_execution	Planner model/provider, purpose, status, request/response hashes, latency, tokens, error and fallback flag.	Shows when and how the LLM Planner was used. No row is needed when Planner is bypassed.
graph_run	invocation ID, thread ID, graph key, status, current node, result, error and timestamps.	Represents the overall LangGraph execution and links it to checkpoints.
graph_node_execution	graph run, node, plan step, attempt, status, small input/output, error and timestamps.	Shows which graph node or plan step ran and where an MVP failure occurred.

13.2 LangGraph-managed tables

Table	Description
langgraph_ckpt.checkpoints	Serialized graph state by thread and checkpoint.
langgraph_ckpt.checkpoint_blobs	Larger channel values used by checkpoints.
langgraph_ckpt.checkpoint_writes	Intermediate node writes.
langgraph_ckpt.checkpoint_migrations	Runtime checkpoint schema version.

13.3 Tables deliberately deferred

A separate idempotency table is not required in the MVP because `agent_invocation.request_id` is unique. A formal audit table, graph transitions, signals, outbox, dead-letter queue, artifact model, agent/workflow registries and all Rule Optimizer business tables are deferred.

13.4 Record creation flow

```

API                -> agent_invocation (QUEUED)
Worker claims      -> agent_invocation (RUNNING)
LLM Planner        -> model_execution (only when invoked)
Plan preparation    -> execution_plan
Graph starts       -> graph_run (RUNNING)
Each node          -> graph_node_execution
Graph checkpoints  -> langgraph_ckpt.*
Graph returns      -> graph_run + agent_invocation (COMPLETED/FAILED)

```

14. First-PR APIs, worker and configuration

14.1 Trigger API

Item	API Contract
Endpoint	POST /v1/agent-inocations
Input	request_id and a small input object; institution/actor comes from trusted backend context.
Processing	Create QUEUED agent_invocation if request_id is new.
Output	HTTP 202 with invocation_id and QUEUED status.
Does not do	It does not wait for the Planner or LangGraph execution.

14.2 Status API

Item	API Contract
Endpoint	GET /v1/agent-inocations/{invocation_id}
Output	Invocation status, plan source, graph status, current node, final result or error.
Source	Read application tables, not LangGraph internals directly.

14.3 Background worker

- Poll one QUEUED invocation.
- Atomically claim it by changing QUEUED to RUNNING only if it is still QUEUED.
- Read simple runtime configuration.
- Call the LLM Planner or load the fixed plan.
- Save the plan and create graph_run.
- Invoke the compiled LangGraph with the plan state and thread_id.
- Save COMPLETED or FAILED status after the graph returns.

14.4 Demonstration stages

Stage	MVP behavior
initialise_context	Place a small initial marker into graph context.
demo_processing	Perform a harmless transformation of demonstration input.
complete	Return a successful demonstration result.

15. First-PR acceptance criteria and test plan

1. POST creates exactly one invocation for a new request_id and returns HTTP 202.
2. A repeated request_id returns or identifies the existing invocation rather than creating another run.
3. The one background worker claims and processes a QUEUED invocation.
4. When Planner is enabled, the LLM returns a structured plan and model_execution is recorded.
5. When Planner is bypassed, the fixed plan is used and no LLM call is made.
6. When the LLM times out or returns an invalid plan, the fixed fallback plan is used and the graph still runs.
7. The plan is stored before LangGraph execution begins.
8. LangGraph executes every demonstration step in order.
9. Each graph node attempt is visible in graph_node_execution.
10. The current node and final graph status are visible through the status API.
11. A simulated worker restart can recover graph state from PostgreSQL checkpoints.
12. No EFRM database, Rule Engine, transaction, case, alert or rule table is accessed.

15.1 Minimum test cases

Test	Expected behavior
Happy path with LLM Planner	LLM plan saved; graph executes; invocation completes.
Planner bypass	Fixed plan saved; no model_execution; graph completes.
Planner timeout	Failed model_execution; FIXED_FALLBACK plan; graph completes.
Invalid stage	Plan rejected; fixed fallback used or invocation fails according to selected MVP policy.
Duplicate request	No second invocation is created.
Node handler error	graph_run and invocation become FAILED with the failed node recorded.
Worker restart	Same graph thread resumes from stored checkpoint or safely restarts according to test configuration.
Status polling	GET shows QUEUED, RUNNING and final status transitions.

16. Implementation sequence and repository structure

1. Project skeleton: Create FastAPI application, settings, database connection and health endpoint.
2. MVP migrations: Create five application tables and add UNIQUE(request_id).
3. Checkpoint setup: Configure langgraph_ckpt connection and run LangGraph checkpointer setup during deployment.
4. Trigger API: Create QUEUED invocation and return HTTP 202.
5. Worker: Implement one polling process and atomic claim.
6. Planner: Implement LLM structured-output request and record model execution.
7. Fixed plan: Add approved JSON plan and fallback loader.
8. Plan validator: Check structure, non-empty steps and stage allow-list.

9. LangGraph: Implement state, load_plan, execute_step, route, finalize and fail.
10. Node records: Record node start, completion and failure.
11. Status API: Return application-level execution state.
12. Tests and README: Cover acceptance scenarios and explain how to run API and worker.

16.1 Suggested repository structure

```

app/
  api/agent_invocations.py
  planner/llm_planner.py
  planner/fixed_plan_loader.py
  planner/validator.py
  graph/state.py
  graph/nodes.py
  graph/routing.py
  graph/builder.py
  graph/handlers.py
  worker/worker.py
  worker/invocation_processor.py
  repositories/
  database/
  settings.py
  main.py
plans/rule_optimizer_mvp.json
tests/

```

17. Deferred work and subsequent PR roadmap

PR Number	Deferred Work
PR 1 - current	Generic Plan-and-Execute MVP only.
PR 2	False-positive trigger/poller, outcome mapping and case verification against approved EFRM read boundary.
PR 3	Case-alert-result-match lineage and source-aware rule/version selection.
PR 4	Rule configuration bundle and historical/current version resolution.
PR 5	Snapshot plan/build, Parquet storage, manifest and data-quality gates.
PR 6	Deterministic health, decay, threshold and overlap analytics.
PR 7	Deterministic strategy, optional LLM explanation, recommendation composition and validation.
PR 8	Recommendation API/UI integration, reviewer decisions, audit, outbox and operational hardening.
Future	Candidate generation, JSON-to-DRL compiler integration, simulator verification and historical backtesting.

17.1 Shared platform hardening deferred from MVP

- Agent/workflow/model/tool version registries and activation bindings.
- Row-level security, full tenant-aware foreign keys and enterprise authorization decisions.
- Outbox publication, dead-letter processing, cancellation signals and worker leases/heartbeats.
- Formal audit partitioning and append-only lifecycle controls.
- Object-storage artifacts, evidence graphs, claim validation and retention/legal hold.
- RAG ingestion, pgvector and document permissions.

18. Open inputs and risks

Open input	Open questions	Open input actions
LLM provider and deployment	Planner implementation needs endpoint, authentication, model and structured-output capability.	Platform/AI architecture approval.
Planner enabled default	Decides whether PR demo uses LLM first or fixed plan first.	Manager decision; configuration, not API user choice.
Fixed plan content	Must define approved demonstration stages and expected order.	Rule Optimizer developer + platform lead.
MVP table DDL approval	execution_plan is proposed; enterprise DB v1.1 may require different naming/dependencies.	Database owner.
Authentication context	institution_id and actor must come from trusted middleware, not arbitrary request JSON.	Platform/API team.
Worker deployment	Need process/container startup, health and database credentials.	DevOps/platform.
Checkpoint DSN and schema	LangGraph runtime tables require dedicated setup and search path.	Database/platform team.
EFRM read contract	Required only after PR 1 for real optimizer data.	EFRM product/database owners.
Metric and outcome policies	Required before analytics can make defensible findings.	Fraud/risk/product owners.

18.1 Main risks

- Scope creep: adding real optimizer tools before the Plan-and-Execute MVP is accepted.
- Confusing LangGraph checkpoints with application reporting tables.
- Letting the LLM choose arbitrary code or tools instead of allowed stage codes.
- Making Planner failure block execution despite the agreed fixed fallback.
- Treating sample database row counts as production sizing evidence.
- Claiming fraud recall or true leakage without confirmed independent outcome labels.
- Assuming deleted historical rules can be reconstructed without source evidence.

19. Decision log

Topic	Final decision
Repository boundary	Agentic Platform is separate from the existing EFRM product and supports multiple agents.
Rule selection	Analyze only rules traced from finalized false-positive cases, not every rule.
Outcome precedence	Final case decision overrides conflicting alert-level decision.
Institution/channel	Configuration-driven and neutral; no pilot institution or channel is fixed.
Data access	Predefined parameterized SQL tools against EFRM read-only boundary for current Phase 1.
Dynamic SQL	Not used by the Rule Optimizer. Restricted safe-query fallback belongs to other controlled use cases.
Analytics owner	Python/SQL calculate official findings; LLM does not calculate metrics.
LLM explanation	Optional and evidence-bounded after deterministic findings.
Planner	LLM-only when invoked, but invocation is optional at workflow level.
Planner selection	Workflow configuration selects mode automatically; normal users do not choose it.
Planner failure	Use approved fixed fallback and continue.
Workflow	LangGraph; no Temporal.
Phase 1 output	Recommendation only; no direct rule write.
RAG	Future policy/document support only; not operational source of truth.
MVP worker	One background worker process; atomic claim supports safe future scaling.
MVP versions	No agent-version or workflow-version resolution in first PR.
MVP database	Five application tables plus four LangGraph-managed checkpoint tables.

20. Handover checklist and glossary

20.1 Before starting implementation

1. Confirm the team accepts the five-table MVP model or supplies the exact approved DDL.
2. Confirm LLM provider/model and secret-management approach.
3. Confirm whether `planner_enabled` defaults to true for the demo.
4. Approve the fixed demonstration plan and allowed stage codes.
5. Confirm FastAPI authentication assumptions and trusted institution context.
6. Create application and checkpoint PostgreSQL connections.
7. Agree on worker startup command and local development process.
8. Create PR acceptance tests before adding future optimizer scope.

20.2 Definition of done for handoff

- A new developer can explain the difference between enterprise target and first-PR MVP.
- They know the worker, not FastAPI, invokes LangGraph.
- They know the Planner is LLM-only when invoked and fixed plan is the bypass/fallback.
- They know the plan becomes LangGraph state and `execute_step` loops through allowed handlers.
- They know which five application tables are required now and which four are runtime-managed.
- They know EFRM data access and all Rule Optimization analytics are deferred to later PRs.

20.3 Glossary

Term	Meaning
Invocation	One API-triggered request to run an agent workflow.
Planner	Optional LLM component that creates a structured ordered plan.
Fixed plan	Approved JSON plan used when Planner is bypassed.
Fallback plan	The same approved fixed plan used after Planner failure.
Background worker	Separate Python process that claims queued work and calls LangGraph.
LangGraph state	Small shared working object passed and checkpointed between graph nodes.
Node	Python function representing one graph step such as load, execute or finalize.
Handler	Approved Python function mapped from an allowed plan stage code.
Checkpoint	Saved LangGraph execution state used for recovery.
Snapshot	Future bounded historical analytical population for one rule/version and policy.
Recommendation	Human-review output; never an automatic production rule change.

Final handover statement Start implementation with the first-PR Plan-and-Execute MVP. Do not add EFRM data tools, optimizer analytics or recommendation logic until the MVP is reviewed and accepted.